

(SET - F261) - Odd
PG-DBDA (Feb 2026 Batch)
Date : 06/04/2026
Module name : JAVA
Time : 10:15 AM to 12:00PM **(40 marks)**

Q1. Class Design & Encapsulation [20 Marks]

Design a BankAccount class that demonstrates proper encapsulation principles:

- a) Declare private fields: accountNumber (String), holderName (String), balance (double).
- b) Provide a parameterized constructor to initialize all fields.
- c) Write getters and setters with validation — balance cannot be set to a negative value (throw IllegalArgumentException).
- d) Implement deposit(double amt) and withdraw(double amt) methods. Withdrawal should fail if balance is insufficient.
- e) Override toString() to return formatted account details.
- f) Write a main() method creating 2 accounts, performing deposits/withdrawals, and printing results.

Q2. Exception Handling — Custom Exceptions & throws Keyword [20 Marks]

Build a simple Student Registration System with custom exceptions:

- a) Create a custom exception class InvalidAgeException (extends Exception) with a constructor that accepts a message String.
- b) Create a custom exception class DuplicateRollNoException (extends Exception) with a constructor that accepts a message String.
- c) Create a class Student with fields: rollNo (int), name (String), age (int).
- d) Create a class StudentRegistration with an ArrayList<Student> and the following methods:
 - i. void registerStudent(int rollNo, String name, int age) — throws InvalidAgeException if age < 17 or age > 30, throws DuplicateRollNoException if rollNo already exists in the list
 - ii. void displayAll() — prints all registered students
 - iii. Student findByRollNo(int rollNo) — returns the student or prints "Not Found"
- e) In main(), demonstrate:
 - i. Register 3 students successfully
 - ii. Try registering a student with invalid age — catch InvalidAgeException
 - iii. Try registering a student with duplicate rollNo — catch DuplicateRollNoException
 - iv. Display all students and search by rollNo